Genetic Algorithms

Boris Velichkov

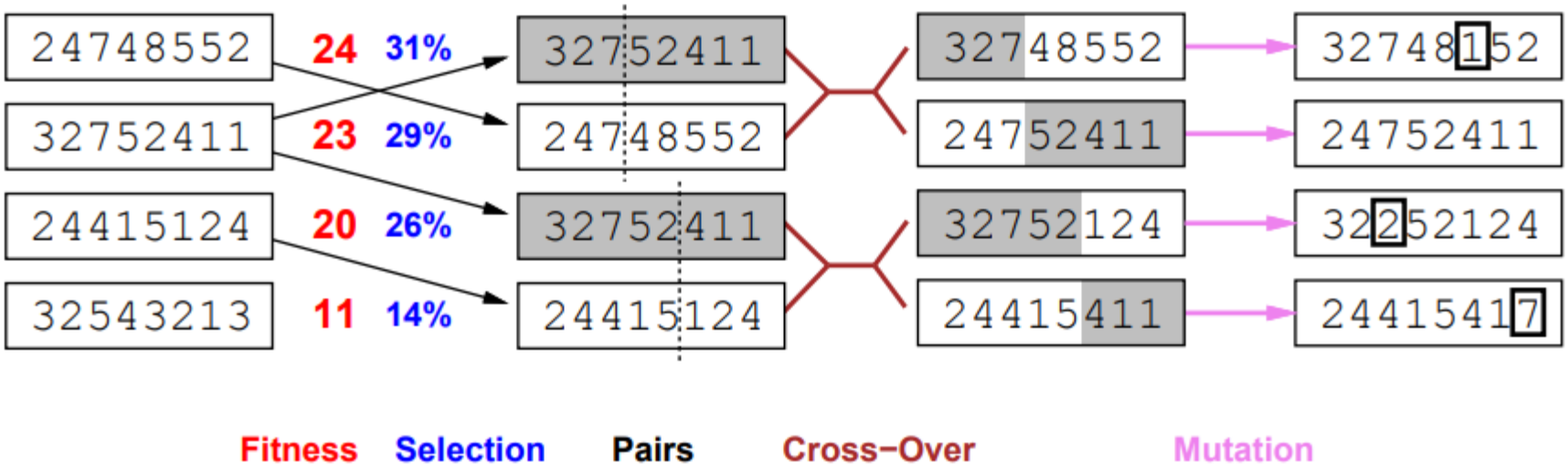
Exercises by Week

1. Introduction
2. **Uninformed *(Blind)* Search**
3. **Informed *(Heuristic)* Search**
4. **Constraint Satisfaction Problems**
5. **Genetic Algorithms**
6. **Games**
7. Introduction to Machine Learning
8. ***k*-Nearest Neighbors**
9. **Naïve Bayes Classifier**
10. **Decision Tree**
11. ***k*Means**
12. **Neural Networks**
13. *Additional Topics, Questions and Homeworks' Presentations*
14. *Additional Topics, Questions and Homeworks' Presentations*
15. *Additional Topics, Questions and Homeworks' Presentations*

Genetic Algorithms

= stochastic local beam search

+ generate successors from *pairs* of states

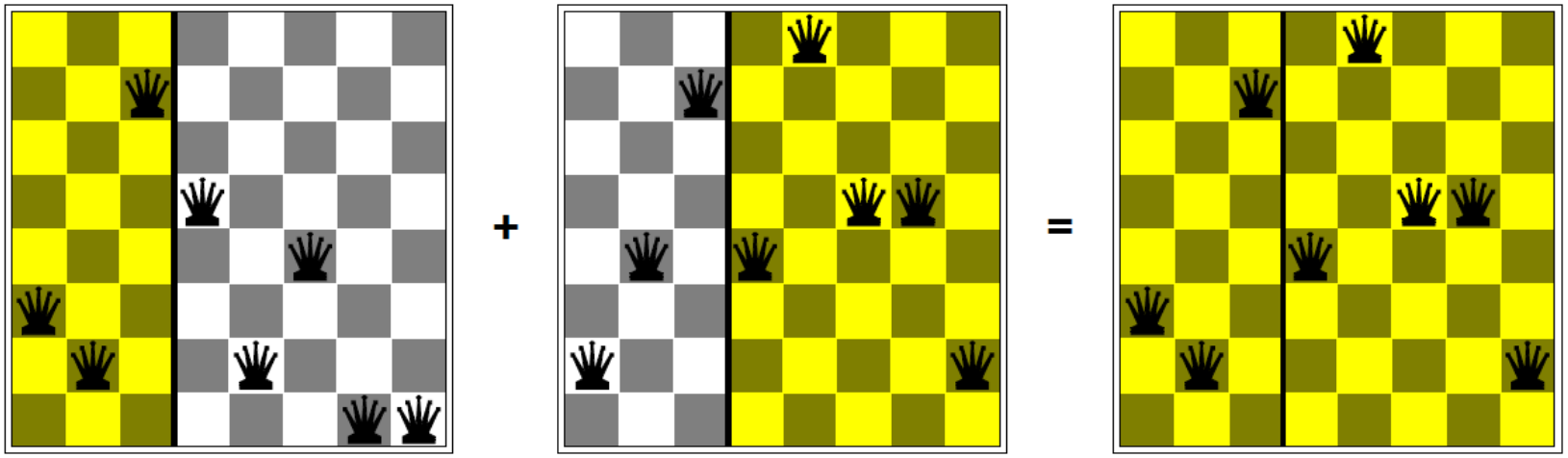


Genetic Algorithms

- In computer science and operations research, a **genetic algorithm (GA)** is a *metaheuristic* inspired by the process of *natural selection* that belongs to the larger class of *evolutionary algorithms (EA)*.
- **Genetic algorithms** are commonly used to generate high-quality solutions to optimization and search problems by relying on bio-inspired operators such as *selection*, *crossover* and *mutation*.
- A **genetic algorithm** is a type of *local search* that *mimics evolution* by taking a *population of strings*, which encode *possible solutions* and combines them based on a *fitness function* to produce individuals that are more fit.

Genetic Algorithms

GAs require states encoded as strings (GPs use programs)
Crossover helps **if substrings are meaningful components**



GAs evolution: e.g., real genes encode replication machinery!

***** *In **N Queens** the population of strings encodes partial solutions.*

GAs: Main Steps

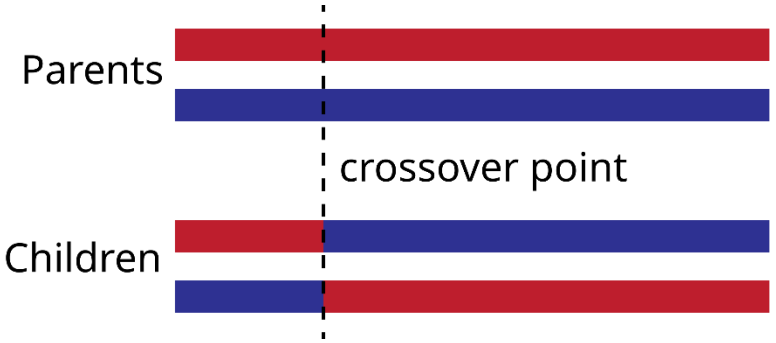
- Initialization
  - Population
- **Selection**
  - Fitness Function
- **Crossover**
- **Mutation**

Selection

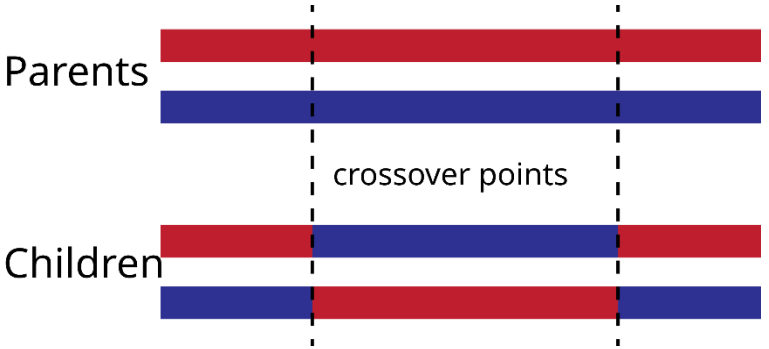
- **Selection** is the stage of a genetic algorithm in which individual genomes are chosen from a population for later breeding (using the crossover operator).
- Methods of **selection**
  - Roulette Wheel Selection (Fitness Proportionate Selection)
  - Tournament Selection
  - Rank-Based Selection
  - Stochastic Universal Sampling (SUS)
  - Elitism
  - Truncation Selection

Crossover

- In GAs, **crossover** is a genetic operator used to vary the programming of a chromosome (or chromosomes) from one generation to the next.
- It is analogous to ***reproduction*** and ***biological crossover***, upon which GA are based.
- **Crossover** is a process of taking more than one parent solutions and producing a child solution from them.



One-point crossover



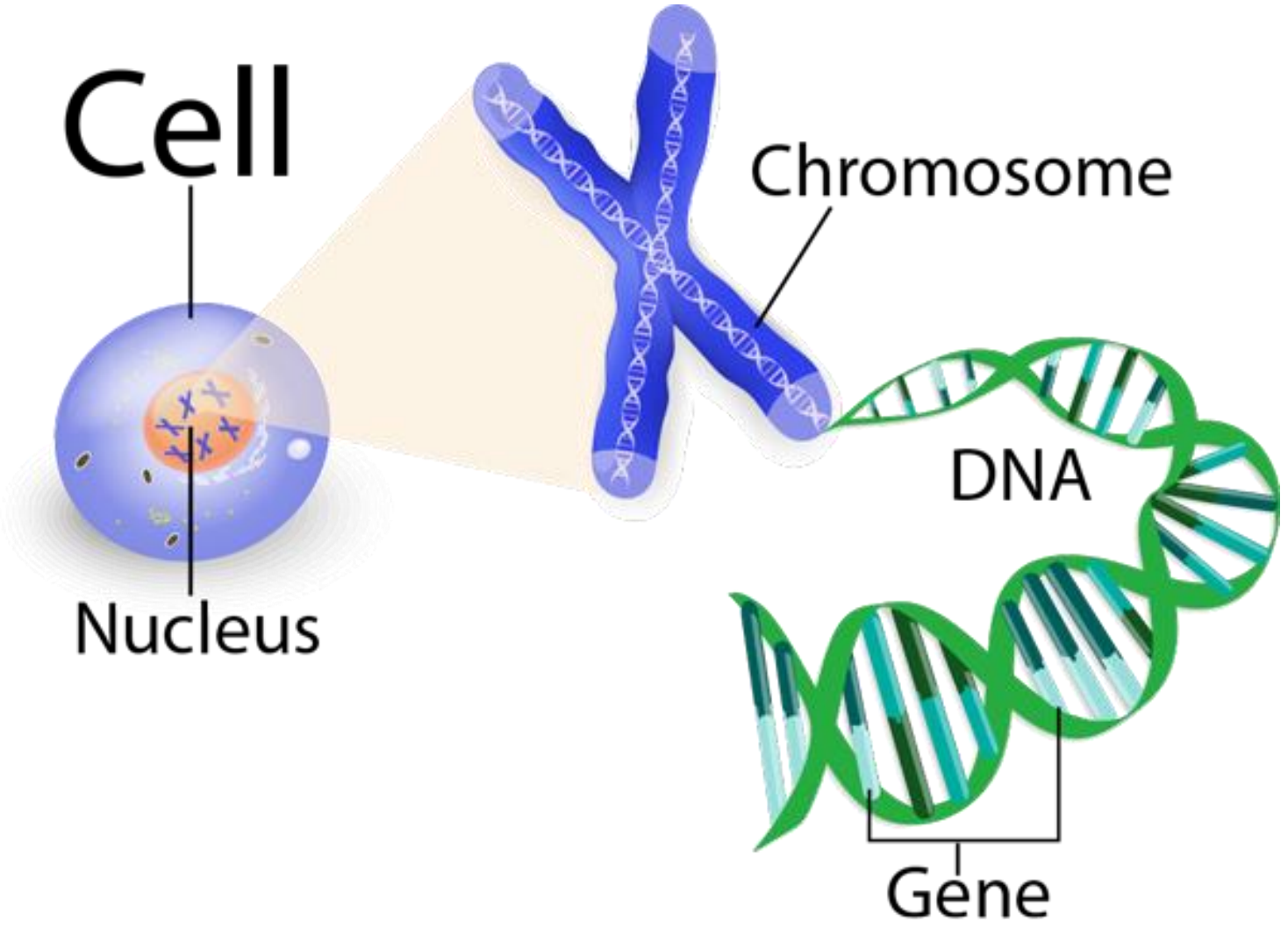
Two-point and k-point crossover

Mutation

- **Mutation** is a genetic operator used to maintain genetic diversity from one generation of a population of genetic algorithm chromosomes to the next.
- It is analogous to ***biological mutation***.
- **Mutation** alters one or more gene values in a chromosome from its initial state.

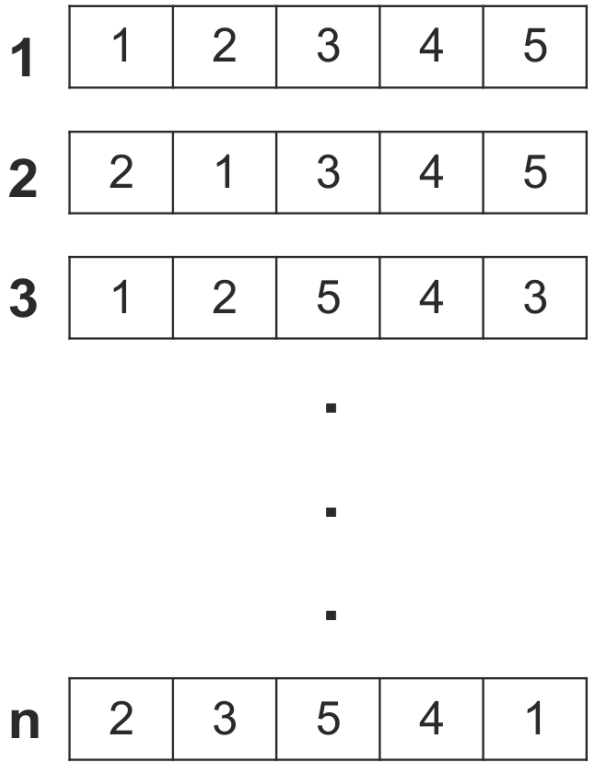
```
1  0  1  0  0  1  0
            ↓
1  0  1  0  1  1  0
```

GAs & Biology

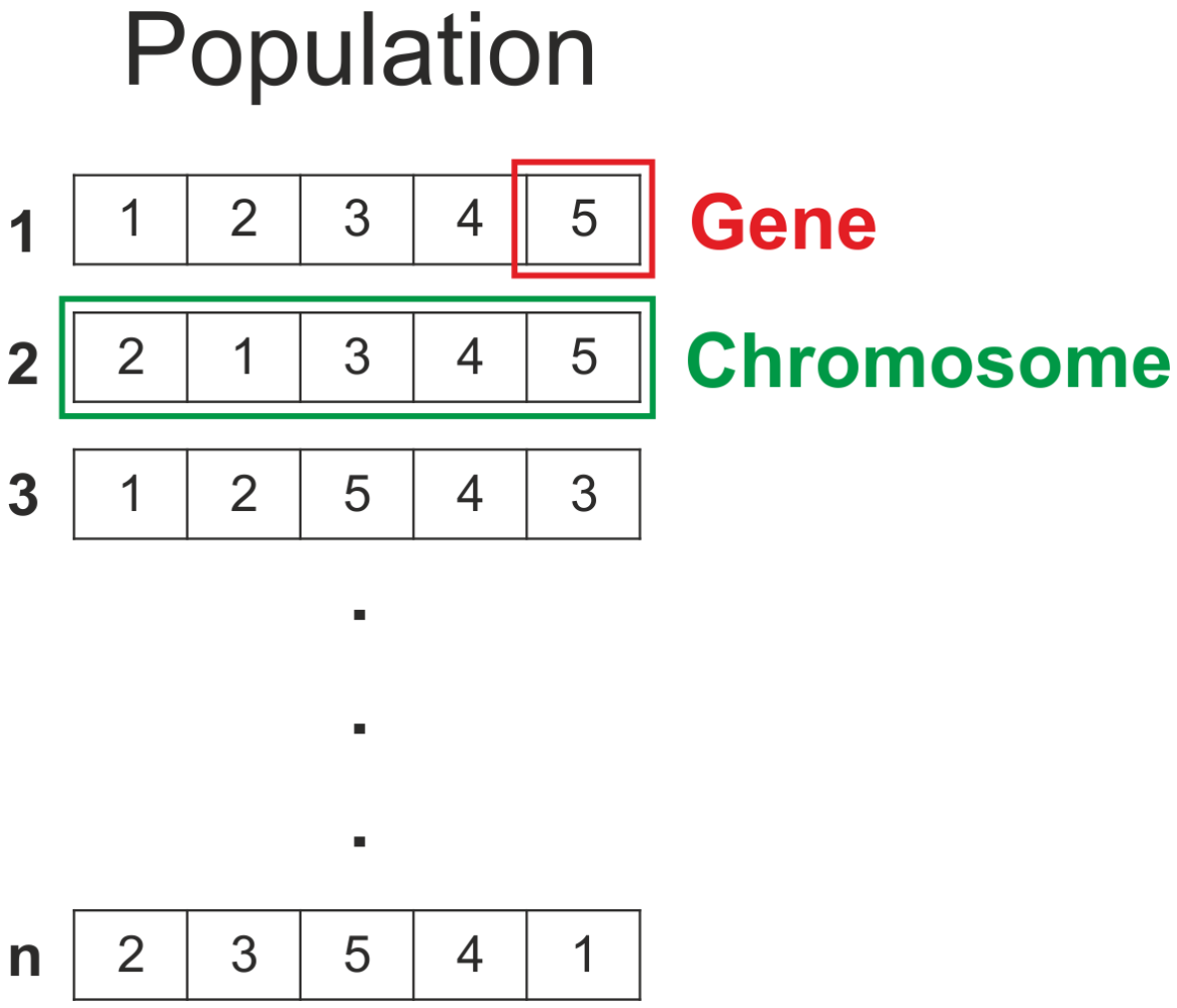


GAs & Biology

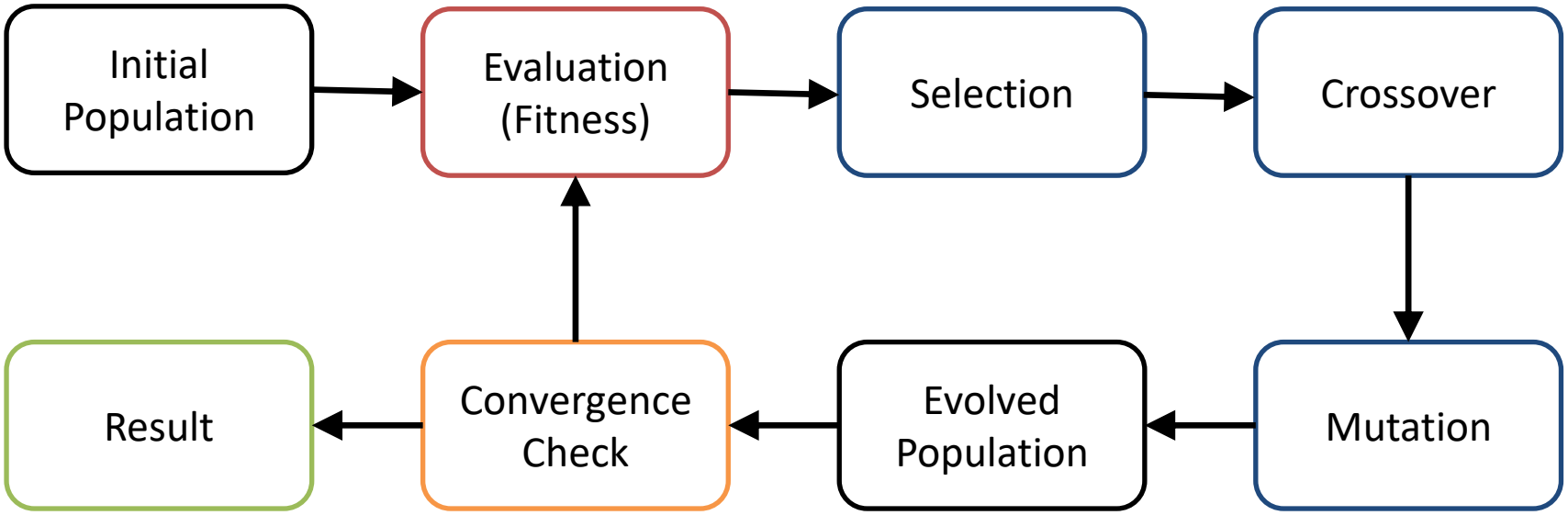
Population



GAs & Biology



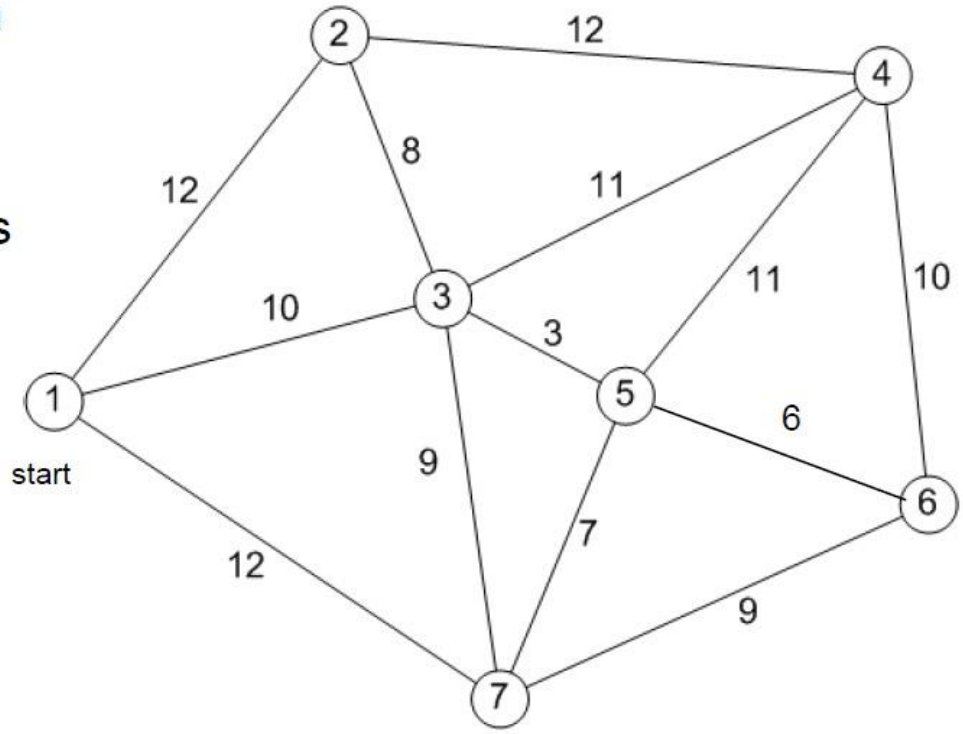
GAs: The „Reproduction“ Cycle



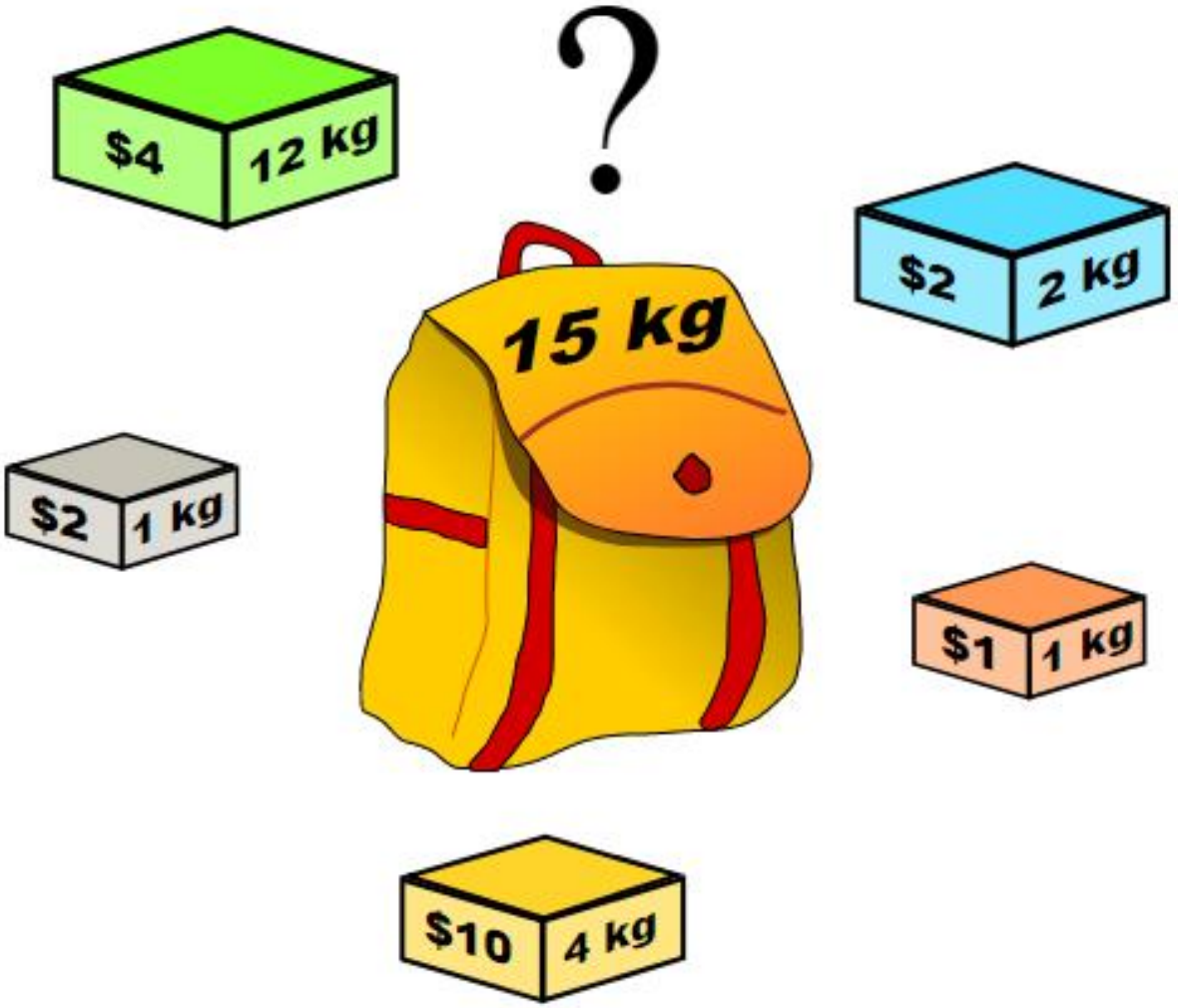
Travelling Salesman Problem

The Traveling Salesman Problem

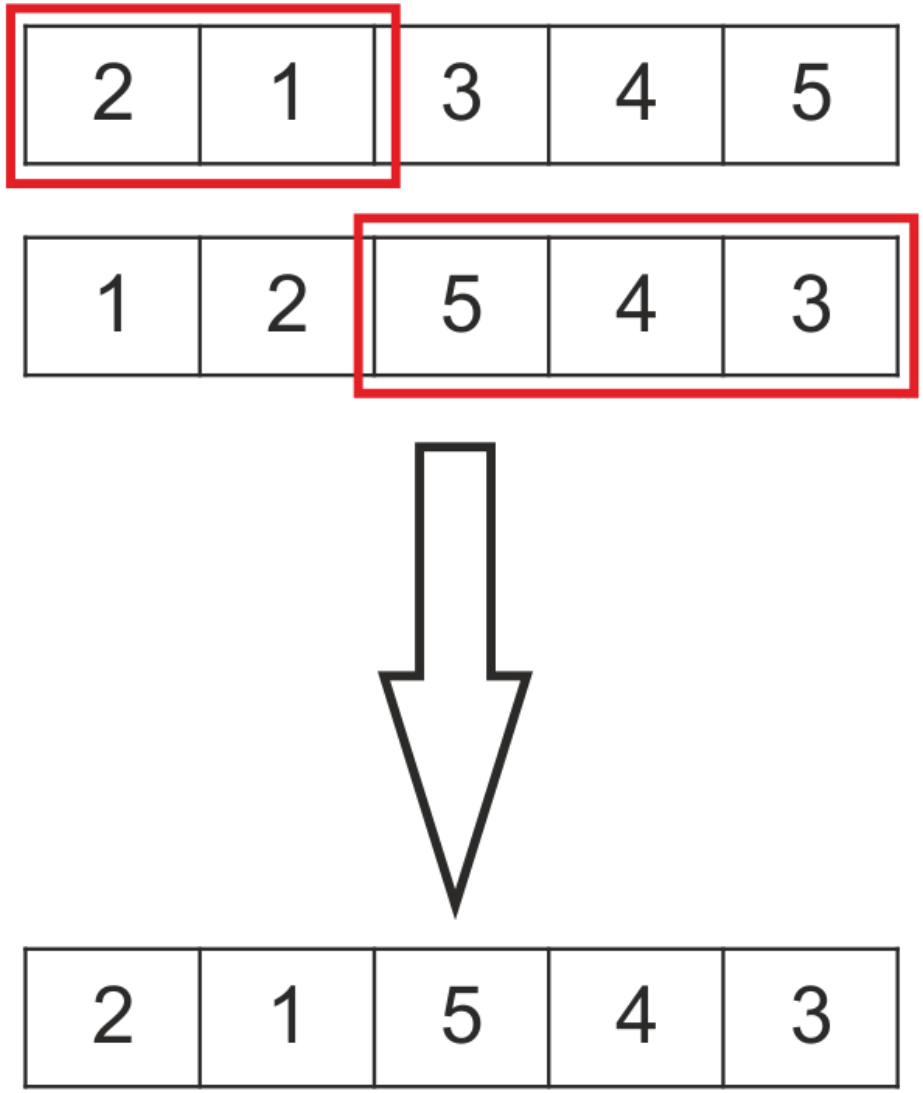
- Starting from city 1, the salesman must travel to all cities once before returning home
- The distance between each city is given, and is assumed to be the same in both directions
- Only the links shown are to be used
- Objective - Minimize the total distance to be travelled



Knapsack Problem



Crossover



One-Point Crossover

1	2	3	4	5

5	4	3	2	1

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3		

5	4	3		

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	_	

5	4	3		

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	5	

5	4	3		

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	5	_

5	4	3		

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	5	4

5	4	3		

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	5	4

5	4	3		

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	5	4

5	4	3	_	

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	5	4

5	4	3	1	

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let's ***i=3***

1	2	3	5	4

5	4	3	1	_

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	5	4

5	4	3	1	2

One-Point Crossover

1	2	3	4	5

5	4	3	2	1

Let‘s ***i=3***

1	2	3	5	4

5	4	3	1	2

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let‘s ***i=3, j=5***

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let's ***i=3, j=5***

		3	4	5		

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let's ***i=3, j=5***

		3	4	5	_	

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let's ***i=3, j=5***

		3	4	5	_	

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let's ***i=3, j=5***

		3	4	5	1	

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let‘s ***i=3, j=5***

		3	4	5	1	_

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let's ***i=3, j=5***

		3	4	5	1	7

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let‘s ***i=3, j=5***

_		3	4	5	1	7

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let's ***i=3, j=5***

6		3	4	5	1	7

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let's ***i=3, j=5***

6	_	3	4	5	1	7

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let‘s ***i=3, j=5***

6	2	3	4	5	1	7

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let‘s ***i=3, j=5***

6	2	3	4	5	1	7

		2	4	3		

Two-Point Crossover

1	2	3	4	5	6	7

7	6	2	4	3	5	1

Let's ***i=3, j=5***

6	2	3	4	5	1	7

1	5	2	4	3	6	7

Partially-mapped Crossover

1	2	3	4	5	6	7

6	3	4	1	2	7	5

Partially-mapped Crossover

1	2	3	4	5	6	7

6	3	4	1	2	7	5

Let‘s ***i=3, j=5***

Partially-mapped Crossover

1	2	3	4	5	6	7

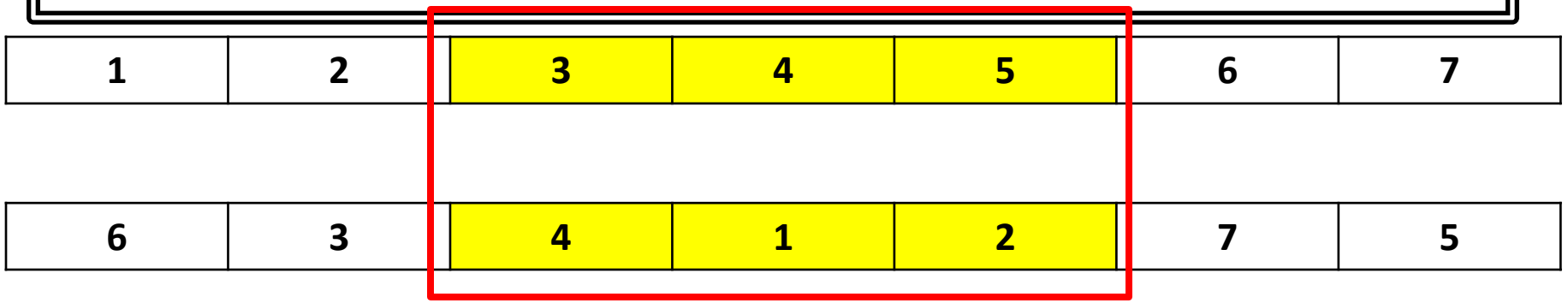
6	3	4	1	2	7	5

Let‘s ***i=3, j=5***

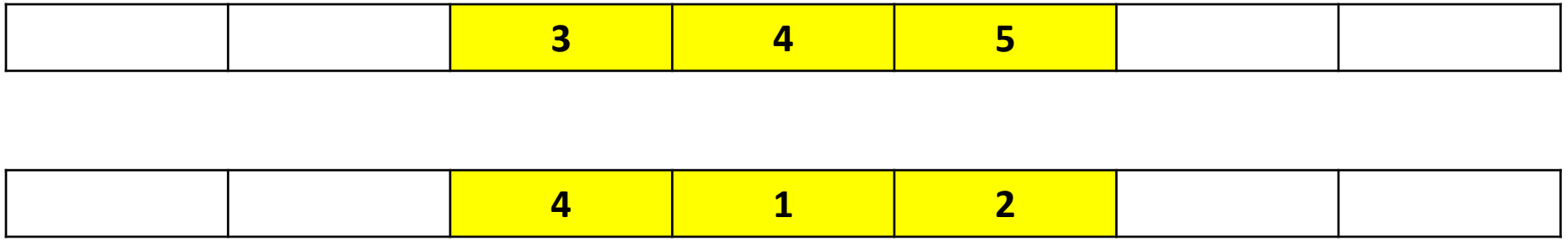
		3	4	5		

		4	1	2		

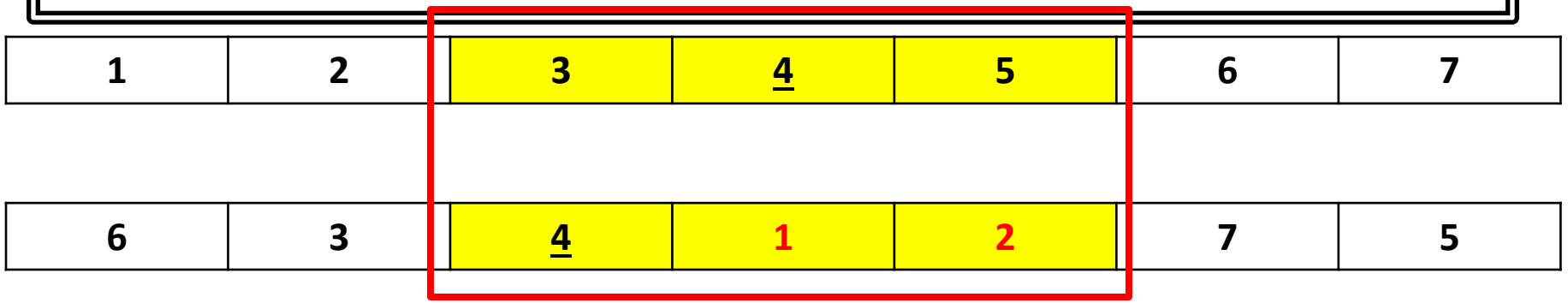
Partially-mapped Crossover



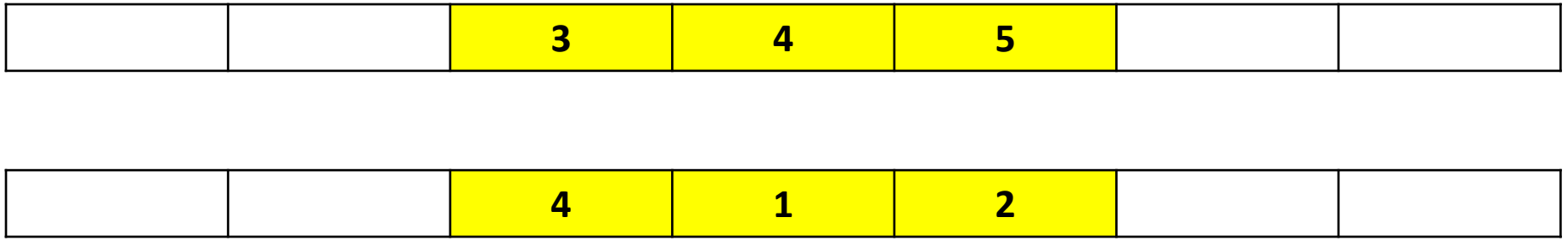
Let's ***i=3, j=5***



Partially-mapped Crossover



Let's ***i=3, j=5***



Partially-mapped Crossover

1	2	3	4	5	6	7

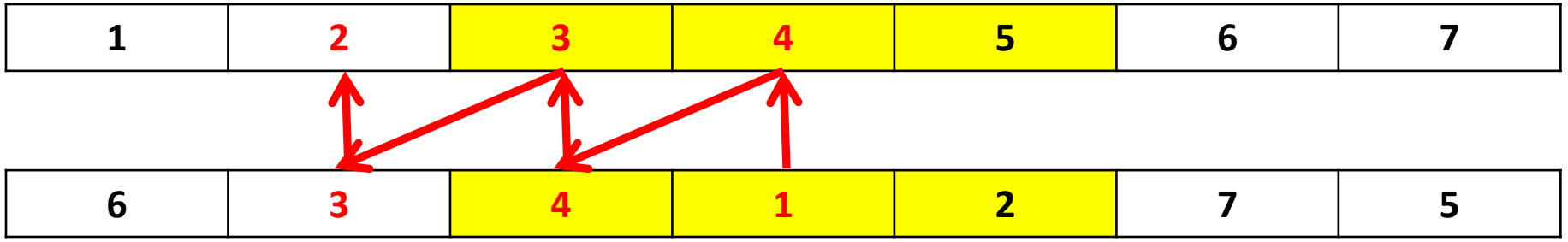
6	3	4	1	2	7	5

Let‘s ***i=3, j=5***

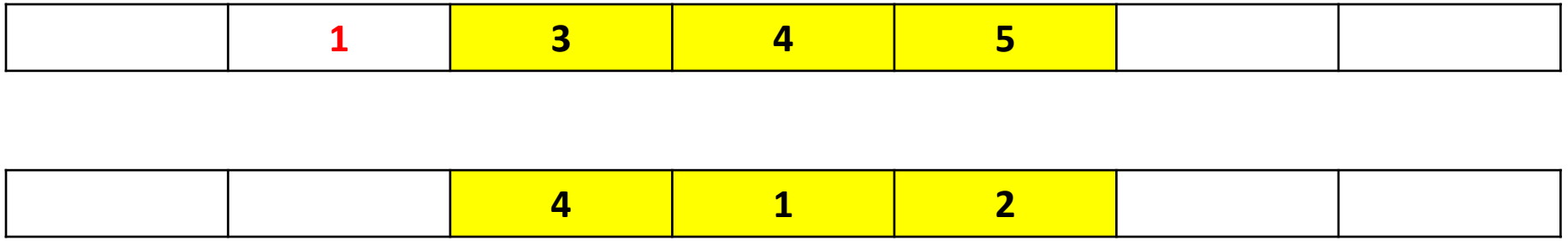
		3	4	5		

		4	1	2		

Partially-mapped Crossover



Let's ***i=3, j=5***



Partially-mapped Crossover

1	2	3	4	5	6	7

6	3	4	1	2	7	5

Let‘s ***i=3, j=5***

	1	3	4	5		

		4	1	2		

Partially-mapped Crossover

1	2	3	4	5	6	7

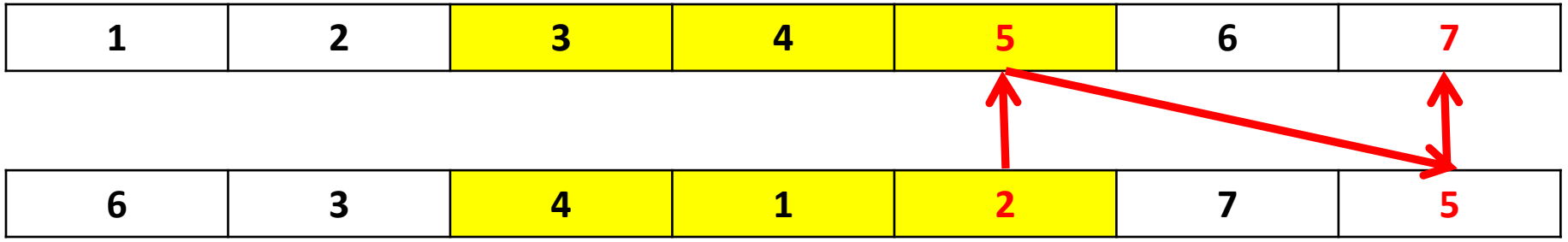
6	3	4	1	2	7	5

Let‘s ***i=3, j=5***

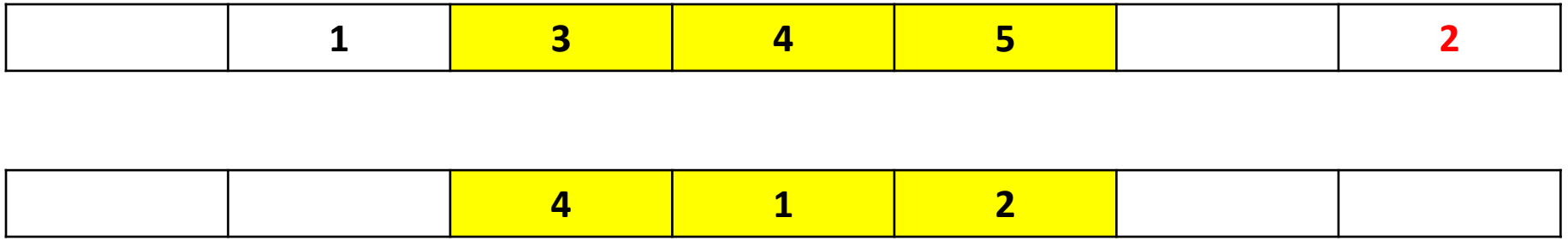
	1	3	4	5		

		4	1	2		

Partially-mapped Crossover



Let's ***i=3, j=5***



Partially-mapped Crossover

1	2	3	4	5	6	7

6	3	4	1	2	7	5

Let‘s ***i=3, j=5***

	1	3	4	5		2

		4	1	2		

Partially-mapped Crossover

1	2	3	4	5	6	7

6	3	4	1	2	7	5

Let's ***i=3, j=5***

_	1	3	4	5	_	2

		4	1	2		

Partially-mapped Crossover

1	2	3	4	5	6	7

6	3	4	1	2	7	5

Let‘s ***i=3, j=5***

6	1	3	4	5	7	2

		4	1	2		

Partially-mapped Crossover

1	2	3	4	5	6	7

6	3	4	1	2	7	5

Let‘s ***i=3, j=5***

6	1	3	4	5	7	2

		4	1	2		

Cycle Crossover

8	4	7	3	6	2	5	1	9	0

0	1	2	3	4	5	6	7	8	9

Cycle Crossover

8	4	7	3	6	2	5	1	9	0

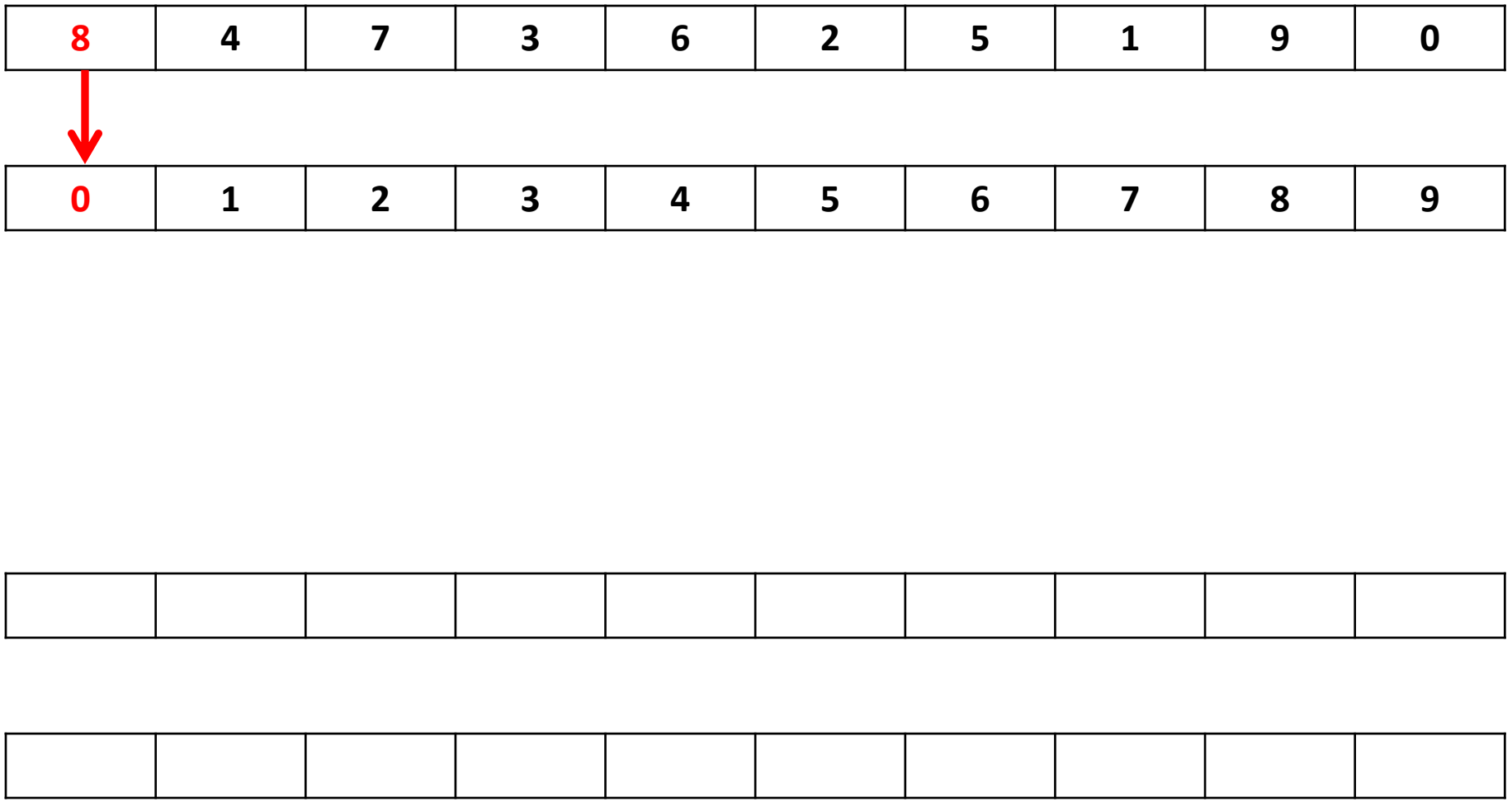
0	1	2	3	4	5	6	7	8	9

Cycle Crossover

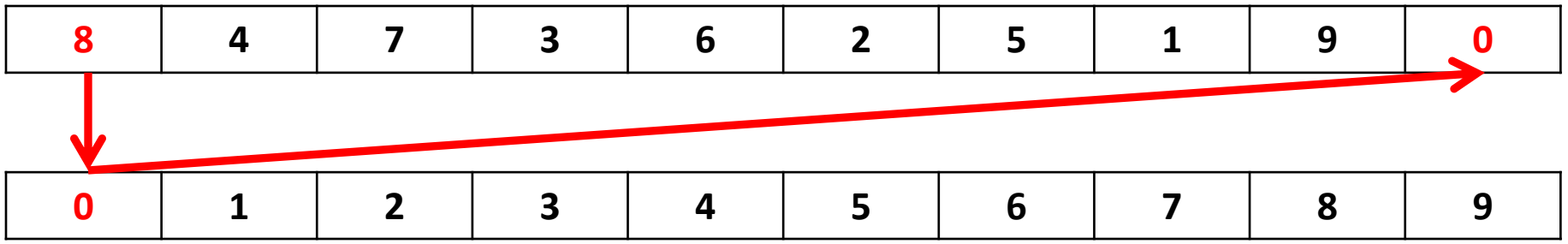
8	**4**	**7**	**3**	**6**	**2**	**5**	**1**	**9**	**0**

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**

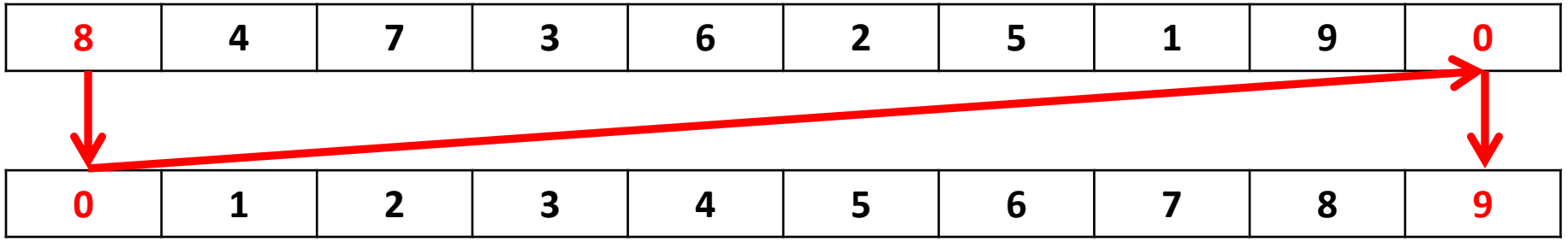
Cycle Crossover



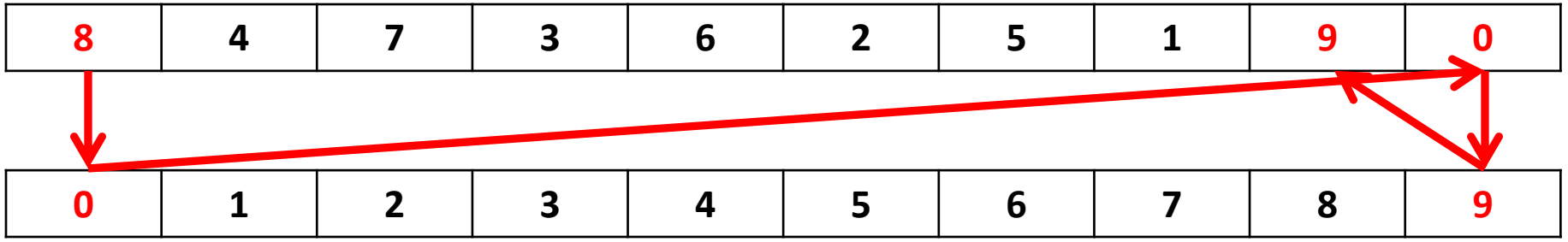
Cycle Crossover



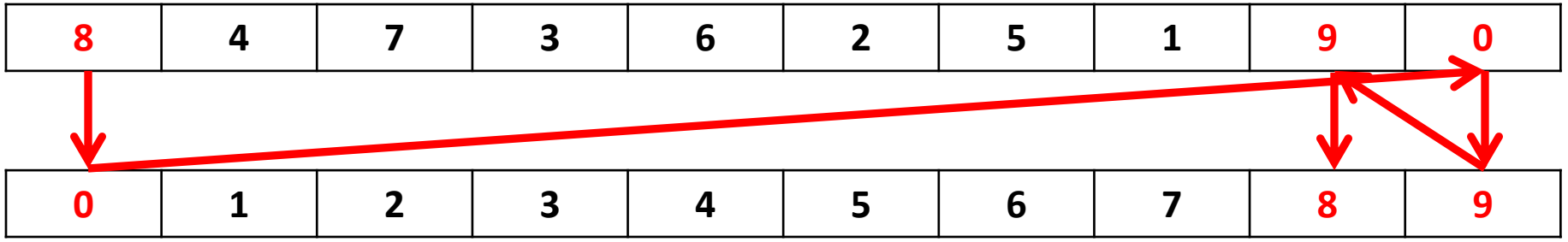
Cycle Crossover



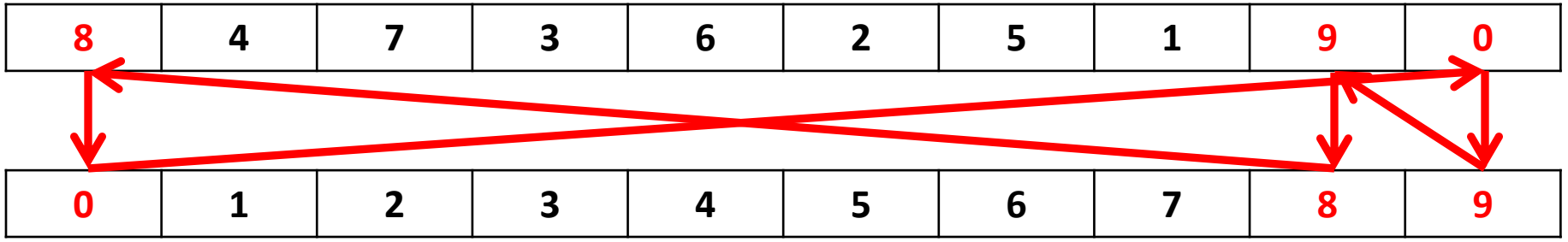
Cycle Crossover



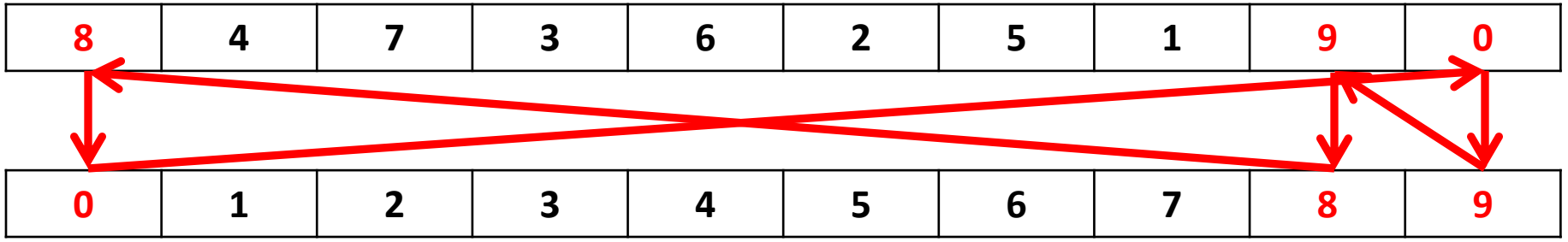
Cycle Crossover



Cycle Crossover



Cycle Crossover



8								9	0

0								8	9

Cycle Crossover

8	**4**	**7**	**3**	**6**	**2**	**5**	**1**	**9**	**0**

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**

8								**9**	**0**

0								**8**	**9**

Cycle Crossover

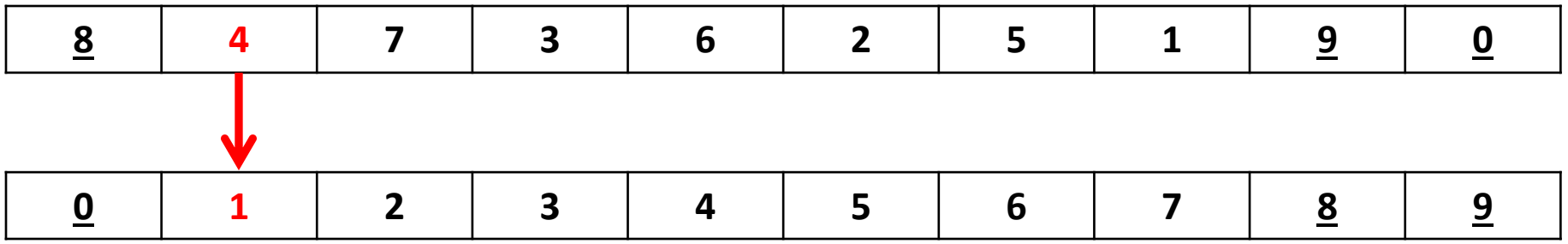
8	**4**	**7**	**3**	**6**	**2**	**5**	**1**	**9**	**0**

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**

8								**9**	**0**

0								**8**	**9**

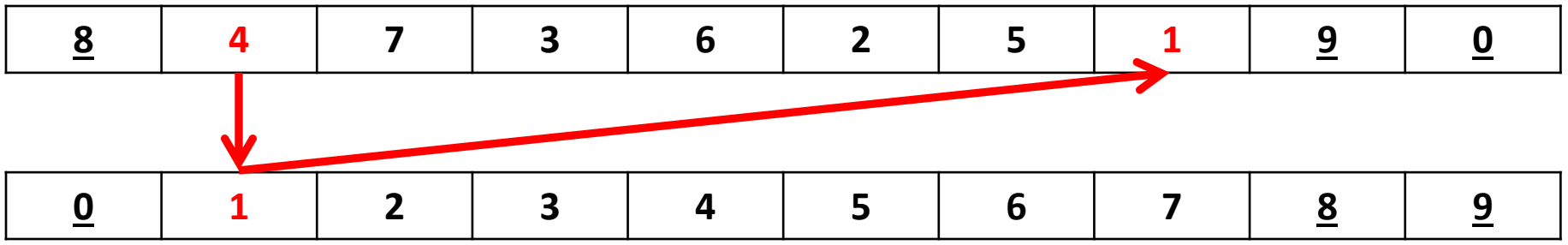
Cycle Crossover



8								9	0

0								8	9

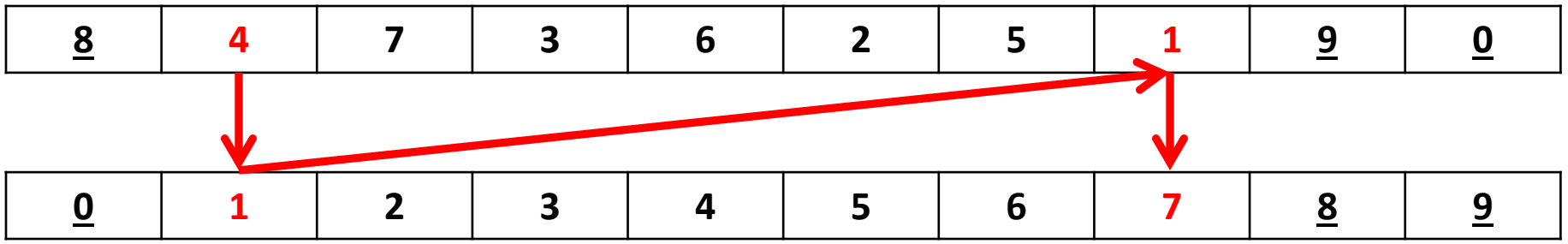
Cycle Crossover



8								9	0

0								8	9

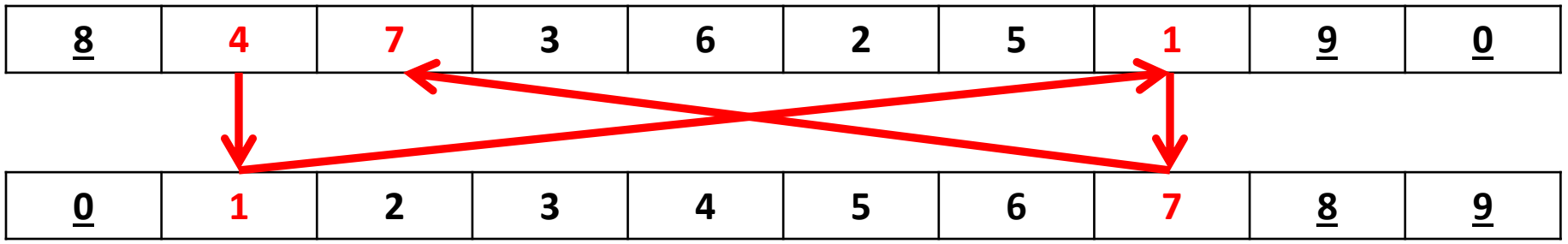
Cycle Crossover



8								9	0

0								8	9

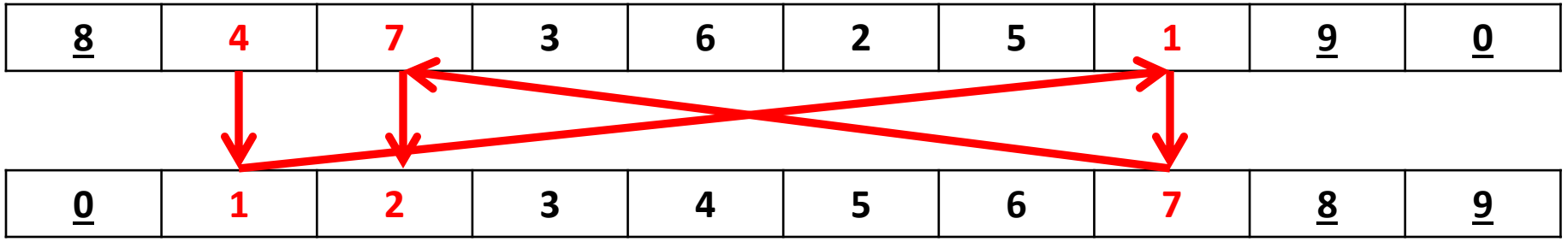
Cycle Crossover



8								9	0

0								8	9

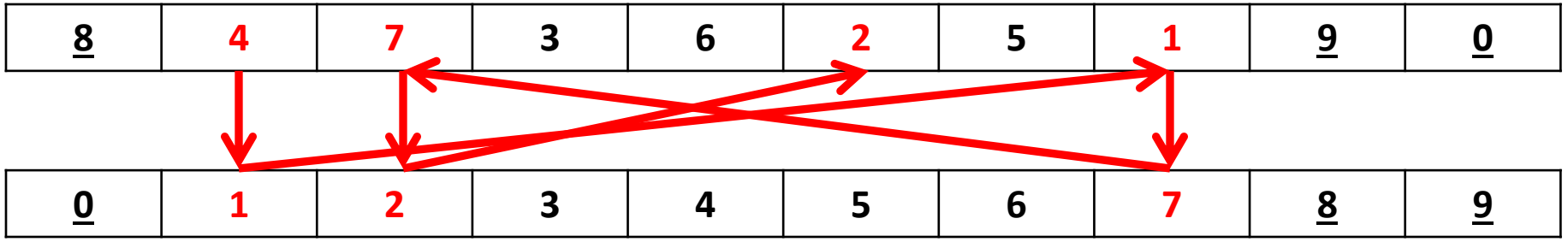
Cycle Crossover



8								9	0

0								8	9

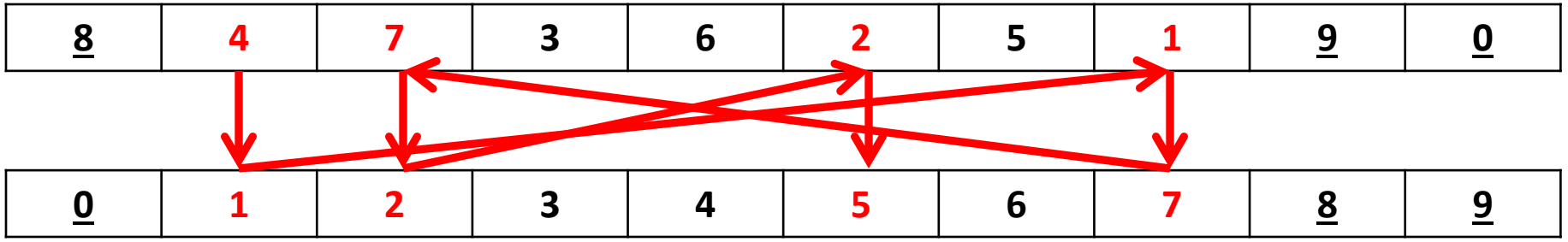
Cycle Crossover



8								9	0

0								8	9

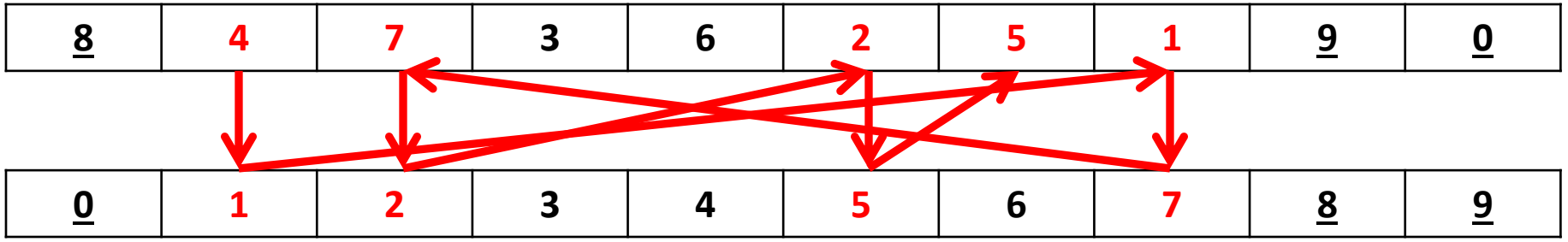
Cycle Crossover



8								9	0

0								8	9

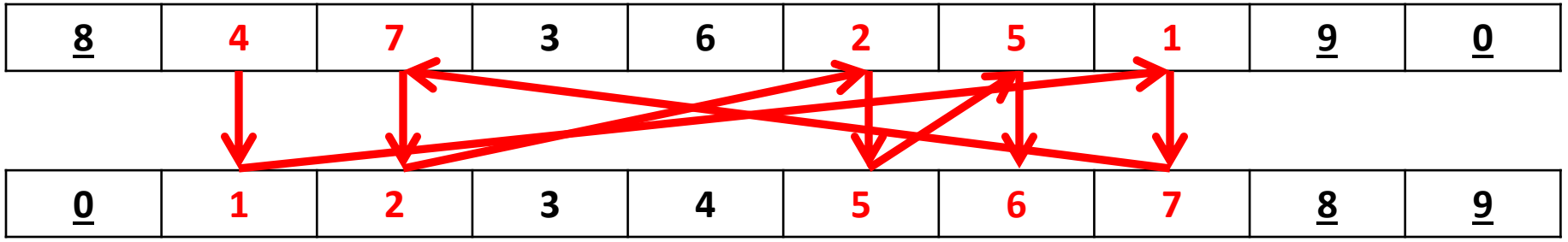
Cycle Crossover



8								9	0

0								8	9

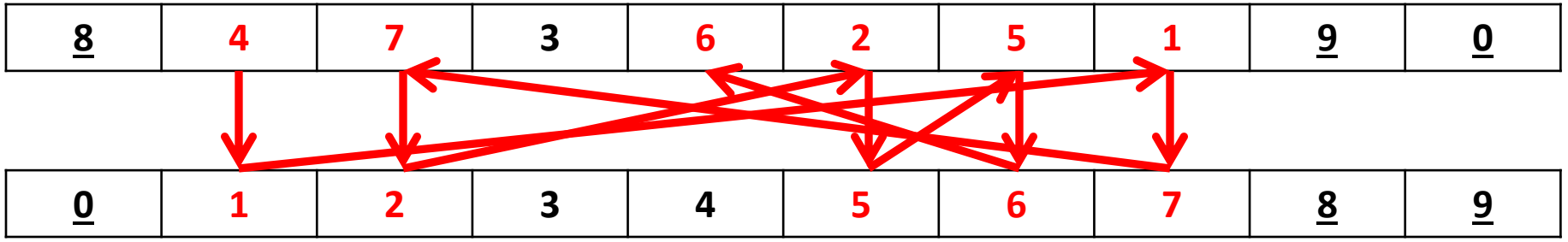
Cycle Crossover



8								9	0

0								8	9

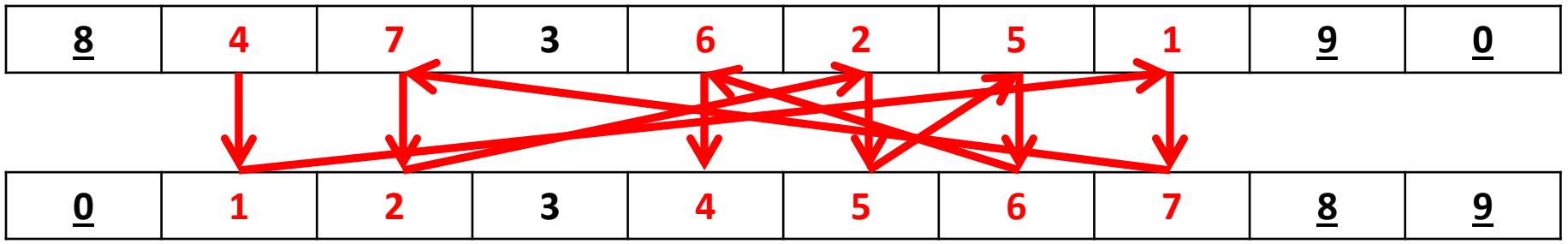
Cycle Crossover



8								9	0

0								8	9

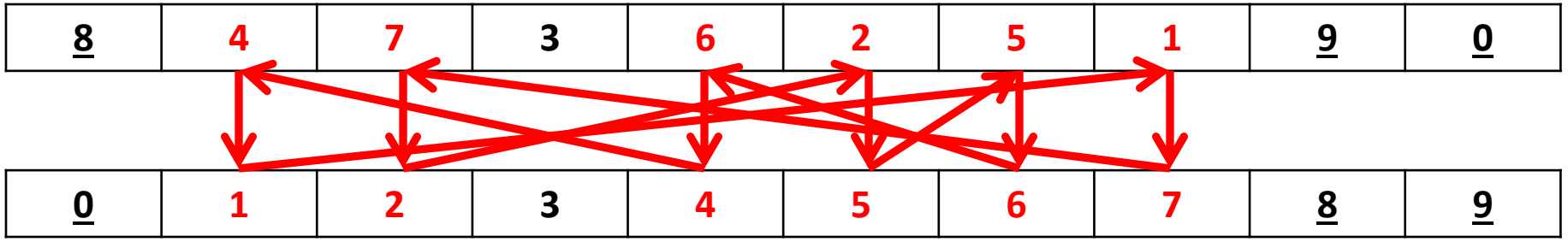
Cycle Crossover



8								9	0

0								8	9

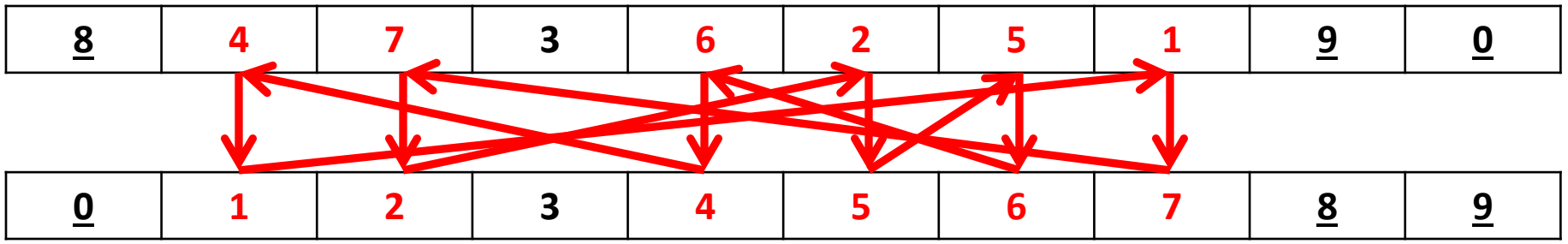
Cycle Crossover



8								9	0

0								8	9

Cycle Crossover



8	1	2		4	5	6	7	9	0

0	4	7		6	2	5	1	8	9

Cycle Crossover

8	**4**	**7**	**3**	**6**	**2**	**5**	**1**	**9**	**0**

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**

8	**1**	**2**		**4**	**5**	**6**	**7**	**9**	**0**

0	**4**	**7**		**6**	**2**	**5**	**1**	**8**	**9**

Cycle Crossover

8	4	7	3	6	2	5	1	9	0

0	1	2	3	4	5	6	7	8	9

8	1	2		4	5	6	7	9	0

0	4	7		6	2	5	1	8	9

Cycle Crossover

8	4	7	3	6	2	5	1	9	0



0	1	2	3	4	5	6	7	8	9

8	1	2		4	5	6	7	9	0

0	4	7		6	2	5	1	8	9

Cycle Crossover

8	4	7	3	6	2	5	1	9	0



0	1	2	3	4	5	6	7	8	9

8	1	2	3	4	5	6	7	9	0

0	4	7	3	6	2	5	1	8	9

Cycle Crossover

8	**4**	**7**	**3**	**6**	**2**	**5**	**1**	**9**	**0**

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**

8	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**9**	**0**

0	**4**	**7**	**3**	**6**	**2**	**5**	**1**	**8**	**9**

Uniform Crossover

1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

Uniform Crossover

1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

Getting a ***gene*** from one of the parents randomly!

Uniform Crossover

1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

Getting a ***gene*** from one of the parents randomly!

Uniform Crossover

1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**1**

2	**3**	**4**	**5**	**5**	**7**	**7**	**9**	**9**	**0**

Getting a ***gene*** from one of the parents randomly!

2									

Uniform Crossover

1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

Getting a ***gene*** from one of the parents randomly!

2	2								

Uniform Crossover

1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

Getting a ***gene*** from one of the parents randomly!

2	2	4							

Uniform Crossover

1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

Getting a ***gene*** from one of the parents randomly!

2	2	4	4						

Uniform Crossover

1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**1**

2	**3**	**4**	**5**	**5**	**7**	**7**	**9**	**9**	**0**

Getting a ***gene*** from one of the parents randomly!

2	**2**	**4**	**4**	**5**					

Uniform Crossover

1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**1**

2	**3**	**4**	**5**	**5**	**7**	**7**	**9**	**9**	**0**

Getting a ***gene*** from one of the parents randomly!

2	**2**	**4**	**4**	**5**	**7**				

Uniform Crossover

1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**1**

2	**3**	**4**	**5**	**5**	**7**	**7**	**9**	**9**	**0**

Getting a ***gene*** from one of the parents randomly!

2	**2**	**4**	**4**	**5**	**7**	**7**			

Uniform Crossover

1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

Getting a ***gene*** from one of the parents randomly!

2	2	4	4	5	7	7	9		

Uniform Crossover

1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**1**

2	**3**	**4**	**5**	**5**	**7**	**7**	**9**	**9**	**0**

Getting a ***gene*** from one of the parents randomly!

2	**2**	**4**	**4**	**5**	**7**	**7**	**9**	**9**	

Uniform Crossover

1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

Getting a ***gene*** from one of the parents randomly!

2	2	4	4	5	7	7	9	9	0

Uniform Crossover

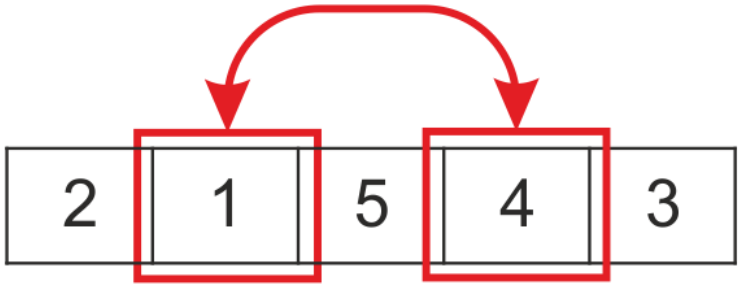
1	2	3	4	5	6	7	8	9	1

2	3	4	5	5	7	7	9	9	0

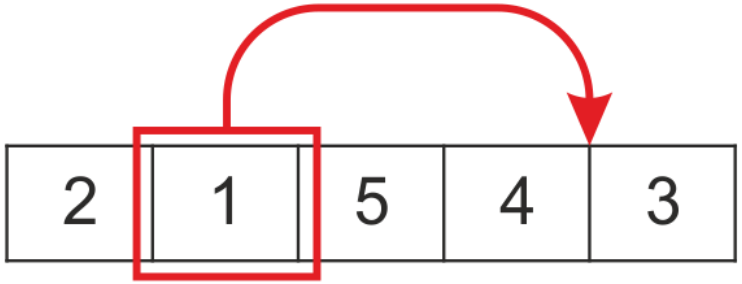
Getting a ***gene*** from one of the parents randomly!

2	2	4	4	5	7	7	9	9	0

Mutation



swap

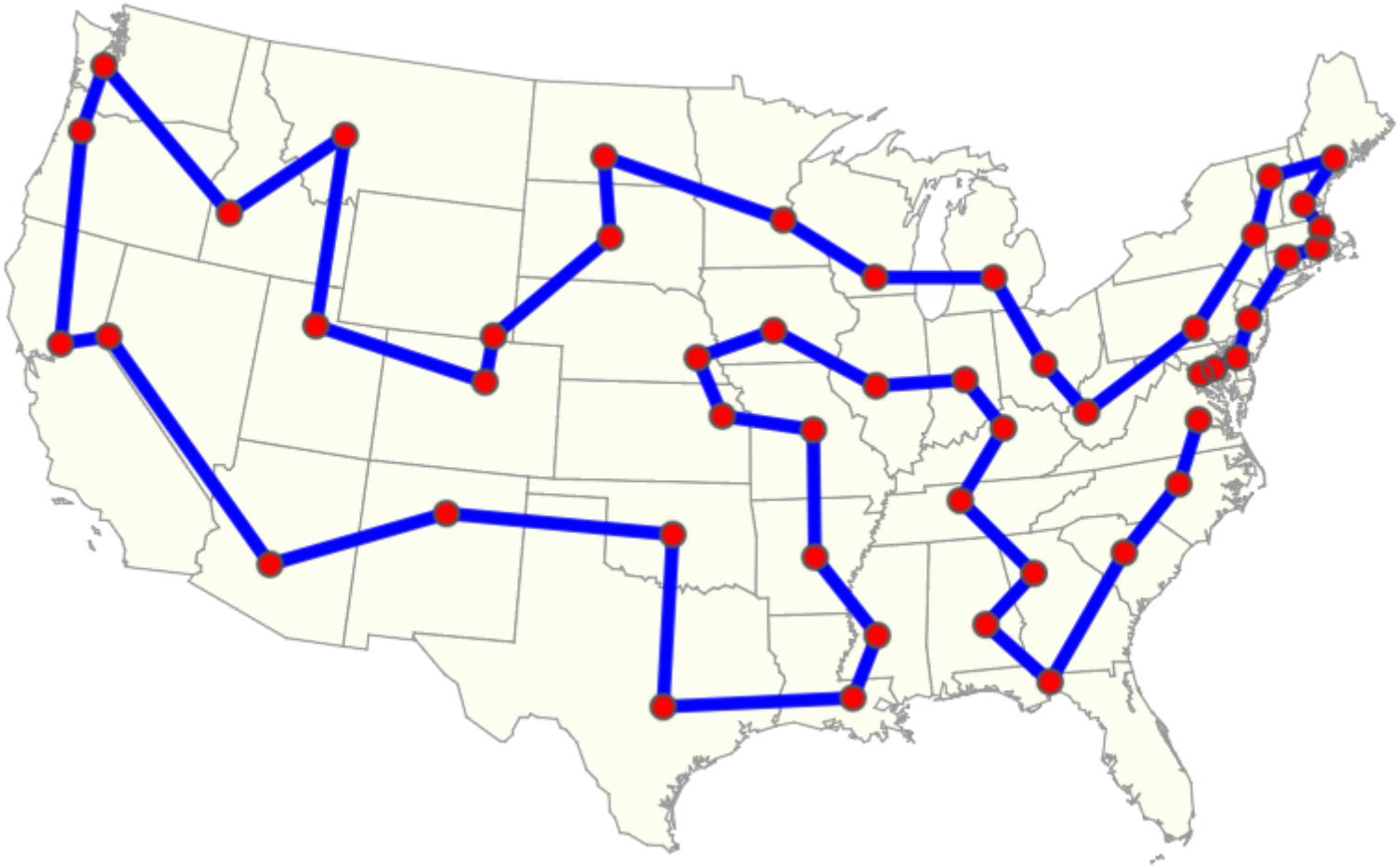


insertion



reverse

Homework: TSP



Homework: TSP

Solve the Traveling Salesman Problem by using a genetic algorithm.

For the purpose of the program is set: ***N*** - integer number (***N <= 100***) - number of points in space (number of cities).

The program generates ***N*** points from a two-dimensional coordinate system at random.

We are looking for the shortest road that passes through each point only once. Print the length of the currently best path in the population for at least five generations:

- **1** - on the 1st generation
- **2**, **3**, **4** - optional
- **5** - after the last generation.

** Use the data in the attached archive "UK_TSP.zip" to check your implementation if it manages to reach the optimal path for the given cities.*

GAs: Pseudocode

```
t = 0
initialize(P(t=0))
evaluate(P(t=0))
while isNotTerminated() do
    P_P(t) = P(t).selectParents()
    P_C(t) = reproduction(P_P)
    mutate(P_C(t))
    evaluate(P_C(t))
    P(t+1) = buildNextGenerationFrom(P_C(t), P(t))
    t = t + 1
end
```